Project Report

Embedded Systems, IS1300

Christoffer Franzén cfranze@kth.se

December 2023

1 Summary

In this report, we will be construct a pedestrian crossing utilizing components of embedded systems, such as a microcontroller from STMicroelectronics and a custom Traffic Light Shield designed specifically for this project. The microcontroller in use is the Nucleo-L476RG, which will be connected to a custom Traffic Light Shield that houses numerous components including LEDs, buttons, switches, and more. Our objective is to establish a fully functional pedestrian crossing.

The pedestrian crossing will have varying task requirements depending on the desired grade. For Task 1, our focus was solely on the upper pedestrian crossing. Task 2 required us to concentrate on the road crossing. In Task 3, we were tasked with implementing both tasks and ensuring their seamless operation together. Additionally, we were required to compose a detailed report outlining our work and the structure of the project.

Contents

1 Summary **1**

2 Methods **3**
2.1 Equipment, Hardware and Software . . . 3
2.2 Hardware . . . 3
2.2.1 Block Diagram . . . 4
2.2.2 GPIO . . . 4
2.2.3 SPI and shift registers . . . 5
2.3 Software . . . 5
2.3.1 Documentation . . . 5
2.3.2 Diagram . . . 5
2.3.3 C programming . . . 6
2.3.4 State machine . . . 6
2.3.5 State machine diagram . . . 6
2.3.6 FreeRTOS . . . 7
2.3.7 Tasks . . . 7
2.3.8 SendSPITask . . . 8
2.3.9 Traffic Lights . . . 8
2.3.10 ToggleLED . . . 9
2.4 Testing . . . 9
2.4.1 Test program . . . 9

3 Results **10**
3.1 Hardware . . . 10
3.1.1 Hardware Performance . . . 10
3.2 Software . . . 10
3.2.1 FreeRTOS implementation . . . 11
3.2.2 Testing Results . . . 11
3.2.3 User Interface . . . 11
3.2.4 Error Handling and Resilience . . . 11
3.2.5 Documentation . . . 11

4 Conclusion **12**
4.1 Compliance with Objectives . . . 12
4.2 Challenges . . . 12
4.3 Possible improvements . . . 13
4.4 Final Thoughts . . . 13

5 References **13**

2 Methods

2.1 Equipment, Hardware and Software

The project's development involved the use of numerous pieces of equipment. On the hardware front, we utilized a microcontroller from STMicroelectronics, specifically the Nucleo-L476RG. Additionally, a custom Traffic Light Shield, designed exclusively for this project, was employed. A USB to USB Mini B cable was used to connect the microcontroller to the development environment on my computer. The software utilized was STM32CubeIDE, a fully integrated IDE from STMicroelectronics, which greatly simplifies the configuration of the microcontroller. I also installed a plugin named EGit in the IDE to be able to use version control for the project. And i used github.com for hosting the a remote private repository. For the creation of the state machine diagram and the hardware and software diagrams, i used Astah with is software modeling tool were you can create state diagrams and block diagrams.

2.2 Hardware

In the course of my project, I utilized only a limited selection of the hardware features offered by the microcontroller and the traffic shield. To organize and visualize the hardware architecture, I employed a block diagram, as shown in Figure 1. This diagram illustrates the various hardware components incorporated in the project and their connections to the microcontroller. For instance, it can be observed that the buttons and switches are interfaced via GPIO. Furthermore, the traffic LEDs are linked through three cascaded shift registers, with the first shift register connected to the microcontroller's SPI interface.

2.2.1 Block Diagram

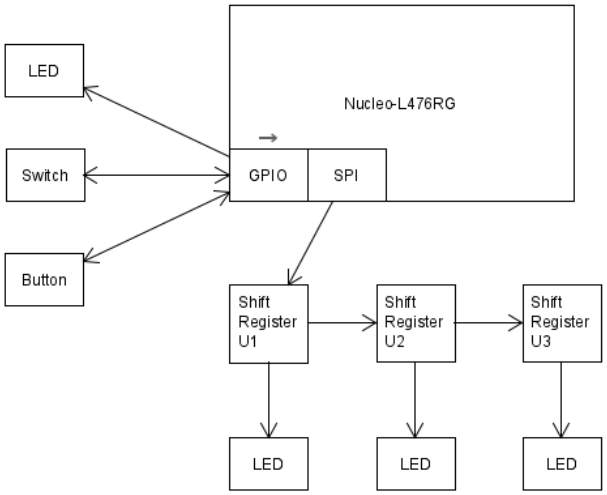


Figure 1: Hardware block diagram

2.2.2 GPIO

In order to facilitate interaction with the buttons, switches, and the top right LED, they were configured as GPIO inputs or outputs. This process required referencing the schematic drawing of the traffic light shield [1] and aligning the various pins of the microcontroller with the corresponding pin of the shield. The inputs were also assigned descriptive names. Subsequently, the GPIO mappings and configurations were executed in the STM32CubeIDE.

GPIO	NUCLEO	Pinout	User Label	GPIO mode
USR_LED1 (60)	NUCLEO CN10 (22)	PB2	LED26	Output
USR_LED2 (64)	NUCLEO CN10 (26)	PB15	LED25	Output
TL1_Car (72)	NUCLEO CN10 (34)	PC4	SW1	Input
TL2_Car (68)	NUCLEO CN10 (30)	PB13	SW2	Input
TL3_Car (66)	NUCLEO CN10 (28)	PB14	SW3	Input
TL4_Car (71)	NUCLEO CN10 (25)	PA10	SW4	Input
PL1_Switch (17)	NUCLEO CN7 (17)	PA15	SW56	Input
PL2_Switch (21)	NUCLEO CN7 (21)	PB7	SW78	Input

Table 1: GPIO Mappings

All the switches are configured with Pull-up, and all the LEDs are configured with No

pull-up and no pull-down.

2.2.3 SPI and shift registers

In the early stages of development, I chose to use the SPI interface for communication with the LEDs. This necessitated the configuration of the SPI3 interface on the microcontroller, as well as the appropriate pins connected to the shift registers. Different pins, such as the SCK and SCTP pins, had different responsibilities.

Once the microcontroller was configured with the correct SPI settings, it was time to create a driver to communicate with the shift registers using SPI. This was achieved by using the HAL_SPI_Transmit function. The process involved first clearing the buffer and disabling the output, then transmitting the data, triggering the registers, and enabling the output. This can be seen in the sendSPIdata function in utilities.c.

As a result, I now have a function that can send three bytes and control the LEDs depending on the data sent.

2.3 Software

The primary program that governs the traffic light system is the state machine. The state machine is tasked with controlling the logic, as well as handling input and output. The state machine manages the LEDs through functions that update the global variable, led data. This global data variable is then read by the sendSPItask, which is responsible for updating the LEDs.

The program's code is divided into different files based on the functionality of the code. Each file has a corresponding header file for organization. The files are divided into separate levels, with the lowest level being utilities.c, which contains all the low-level code for tasks such as communicating with the LEDs and handling read inputs. These functions are then utilized in the higher-level code in the state machine, where much of the low-level code has been abstracted away for the application code.

The state machine is essentially a large loop that runs repeatedly. It employs numerous timers to monitor the time between changes, such as pedestrian crossings and delays between light switches. A high level of the software architecture is depicted in Figure 2.

2.3.1 Documentation

In order to document the code, I adhered to the Doxygen standard. I ensured that each file included a header that described the file and its functions. Additionally, I made certain that every method was accompanied by documentation that outlined the function, its parameters, and the data it returns.

2.3.2 Diagram

Before starting to program the code for the traffic light i begun to make a high level design diagram of how i wanted to software to be structured. I wanted to create different appstraction layers so that it would be easy to for example turn of a light in the state

macine. This necicitated the creration of layers in the software archetercure as seen in Figure 2.

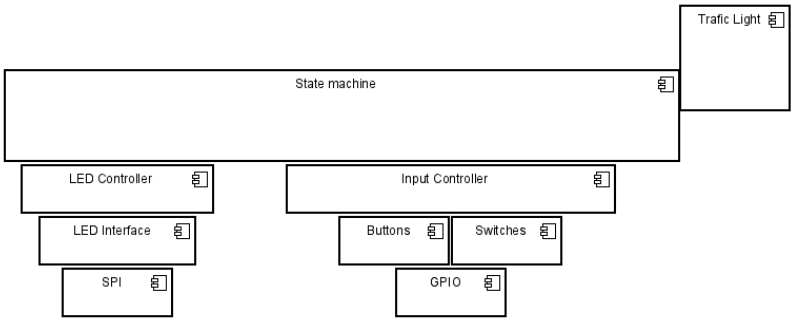


Figure 2: Software layered diagram

2.3.3 C programming

The programming language utilized in the project was C, which is a low-level language commonly employed in the development of embedded systems. During the development process, I used structures to create data types, thereby simplifying the creation of the traffic light system.

2.3.4 State machine

Buy fare the most complicated part of the project was the creating of the state machine. The state machine responsibility was to have all the logic of the application and implement all the task of the project requirement with was quite meany if you both wanted to implement task 1and task 2.

The state machine manages different states of the traffic light and pedestrian signals based on various conditions, such as the time elapsed sine the last state change pedestrian button presses, and the status of car sensors.

2.3.5 State machine diagram

At the beginning of the state machine's creation, I developed a basic state diagram to model the functionality of the state machine and the various transitions between the states see Figure 3. This approach was effective when implementing the initial task. However, when it was necessary to combine all the tasks, I did not update the state machine diagram due to it size. Despite this, the state diagram proved to be useful in the early stages of development, as it provided a visual representation of how to structure the code.

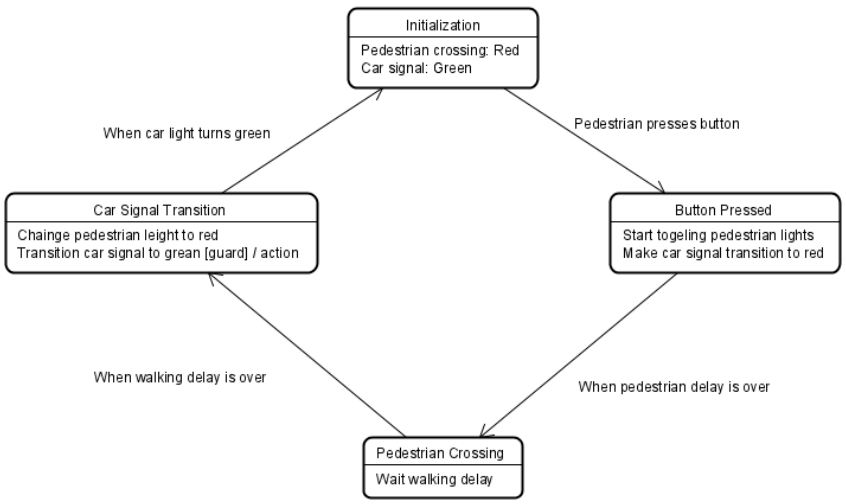


Figure 3: State machine diagram

2.3.6 FreeRTOS

In the course of my project, I decided implement a real-time operating system, specifically FreeRTOS. FreeRTOS offers the capability to establish multiple tasks that can operate concurrently, even within a single-core system. This feature enhances flexibility and enables the execution of simultaneous tasks. In this particular scenario, we could divide the functionality of the traffic light system into several tasks, allowing the traffic light to exist in multiple states. FreeRTOS also provides additional features such as timers, semaphores, and memory management. However, many of these features may not be utilized in this project.

2.3.7 Tasks

My initial strategy was to divide the traffic light system into two distinct tasks or threads, with these two tasks constituting the entire traffic light system. However, I realized that this approach would be quite complex, necessitating communication between the different tasks using semaphores and other mechanisms. Consequently, I decided to use only one task for the traffic light and another task for hardware communication, specifically with the traffic light connected via the SPI interface.

The reasoning behind creating a specific task solely for communication with the SPI interface was to structure the program effectively, allowing for improved task management and isolation. The main program now only needs to update a global variable to change the LEDs, and then the SPI task sends the variable data to the SPI interface, light up the appropriate LEDs.

The following are the tasks created for this project. I also adjusted the priority of the SendSPITask. The defaultTask is not utilized in the traffic light system, but it is employed to execute the test program when the test variable is set to true. Otherwise, the defaultTask deletes itself upon startup.

Task Name	Priority	Stack Size	Entry Function	Allocation
defualtTask	osPriorityNormal	128	StartDefualtTask	Dynamic
SendSPITask	osPriorityHigh	128	SendSPIdataTask	Dynamic
TrafficLight	osPriorityNormal	128	TrafficLightTask	Dynamic
ToggleLED	osPriorityHigh	128	ToggleLEDTask	Dynamic

Table 2: FreeRTOS Tasks and Queues configuration

2.3.8 SendSPITask

As the task name implies this task is responsible for comunicating the the SPI interface and controlling the traffic light leds. The task is quite simple it reands a global variables and chech if the value of the global varialbe has change and if so it updates the traffic light leds. This task is rund periodicly every 50 ms and has a high priority meaning it takes precident over the odther tasks.

2.3.9 Traffic Lights

As outlined in the preceding chapters, the traffic light LEDs are connected using three shift registers. To communicate with these shift registers, I employ the SPI interface on the microcontroller. This also necessitated the mapping of LEDs to data. For instance, the LED labeled LED1 on the traffic light shield is connected to the third shift register at the final position, position 8. To illuminate this light, we need to send a 0x00, 0x00, 0x01, which is an array of three bytes with the first bit as 1 in the last byte. This is how the LEDs are mapped in the program. Some LEDs, such as 23 and 24, are connected to the same connection and can only be activated simultaneously, hence they share the same memory address.

```
#define LED1 0x00, 0x00, 0x01
#define LED2 0x00, 0x00, 0x02
#define LED3 0x00, 0x00, 0x04
...
#define LED22 0x00, 0x10, 0x00
#define LED23 0x00, 0x20, 0x00
#define LED24 0x00, 0x20, 0x00
...
#define ALL_LEDs 0xff, 0xff, 0xff
#define NO_LEDs 0x00, 0x00, 0x00
```

2.3.10 ToggleLED

In order to toggle the blue pedestrian LEDs at a specific frequency, I chose to create a separate task for this purpose. This task is configured to operate at the specified toggle frequency. It is executed at every toggle frequency interval and runs the function toggleLED(ledID);, which toggles the pedestrian blue light, thereby achieving the desired outcome. Initially, the task is suspended when the program starts, but it resumes when a user presses the pedestrian button. The task is then suspended again when the blinking LED is no longer required, ensuring that the task only runs when necessary.

2.4 Testing

During the development of the pedestrian crossing embedded system a test driven approach to development was taken. The process adhered to the cycle of Test-Driven Development, which involves the following sequence of steps: initially, a test is added; subsequently, the test is executed to ensure it fails; following this, the necessary code is written; the test is then run repeatedly until it is successfully passed; finally, the code is refactored and tidied up. This systematic approach ensures the robustness and reliability of the developed system.

2.4.1 Test program

To execute the test program, it is necessary to define the RUN_TEST_PROGRAM macro in the freertos.c source code file. This can be accomplished by removing the line where RUN_TEST_PROGRAM is undefined.

```
/* USER CODE BEGIN PD */
#define RUN_TEST_PROGRAM
#undef RUN_TEST_PROGRAM
/* USER CODE END PD */
```

The test program comprises various tests that examine the functionality of the created functions to ensure they operate as expected. As the tests are not automated, the developer is required to confirm that the tests pass by running the program and verifying that the output matches the expected results from the test. The following are the tests created for this project.

```
void test_program(void) {
        test_lightUpLEDs();
        test_traficLights();
        test_GPIOLeds();
        test_GPIOInputs();
        test_ledOn();
        test_trafficLightStateMachine();
}
```

Certain tests, such as the GPIO inputs test and the traffic state machine test, require inputs from the developer. To proceed from one test to the next, press both pedestrian signal buttons simultaneously.

For instance, to test the lights, I developed three separate tests, each testing the light in different ways. The code includes documentation detailing what each test does and what the expected test output should be. For example, the test_lightUpLEDs function flashes LED1, 6, 9, and 13 three times. This is what the developer needs to verify in order to mark the test as passed. The function used the lightUpLEDs functions that manipulate the data variable that the sendSPIdata function reads and then updated the LED's.

3 Results

3.1 Hardware

The hardware integration has been a critical aspect of the project, contributing significantly to the overall functionality of the pedestrian crossing system. Despite encountering minor issues, such as lights flickering when certain parts of the microcontroller and traffic light shield are touched, the hardware's overall performance has been commendable. This phenomenon of flickering lights may be attributed to potential interference, and while it does not impact the system's regular operation, it remains an aspect to be investigated further for optimal user experience.

3.1.1 Hardware Performance

The performance of the embedded hardware has met expectations throughout the development phase. The microcontroller, specifically the Nucleo-L476RG, has demonstrated its capabilities by efficiently handling the diverse tasks required for the pedestrian crossing system. The custom Traffic Light Shield, designed exclusively for this project, has proven reliable in housing and facilitating the operation of various components, including LEDs, buttons, and switches.

The absence of significant limitations in the features or performance of the microcontroller underscores its suitability for embedded systems applications. The robustness of the hardware forms a solid foundation for the reliable operation of the pedestrian crossing system.

3.2 Software

The software component of the pedestrian crossing system plays a pivotal role in orchestrating the various functionalities, ensuring seamless interaction between hardware elements. The software's performance has been evaluated through rigorous testing, and its responsiveness and reliability contribute significantly to the overall success of the project.

3.2.1 FreeRTOS implementation

The integration of FreeRTOS in the pedestrian crossing system enabled efficient multitasking. Tasks, such as controlling traffic lights and managing SPI communication, ran concurrently with defined priorities. Synchronization mechanisms, including semaphores, ensured that only one task could use the SPI interface at a time. Real-time responsiveness was achieved through timed intervals managed by FreeRTOS timers.

Challenges, such as flickering LEDs during rapid task execution, were mitigated by updating LEDs only on data changes. In conclusion, FreeRTOS significantly enhanced task management and concurrency in the pedestrian crossing project.

3.2.2 Testing Results

A comprehensive testing strategy has been implemented to validate the functionality of the software components. The test program, covering various aspects such as LED control, traffic light states, and GPIO interactions, has been instrumental in identifying and rectifying potential issues. The effectiveness of the test program is reflected in its ability to ensure that critical functionalities operate as intended.

While the current test program provides a robust evaluation of core features, it's essential to acknowledge that the complexity of the entire system may introduce challenges that go beyond the scope of automated testing. Hence, manual testing and user feedback remain crucial aspects of ensuring the system's overall reliability.

3.2.3 User Interface

The user interface, comprising buttons and switches, has proven to be intuitive and reliable during testing. User inputs are consistently detected and processed accurately, contributing to the overall user experience. The absence of encountered problems with the user interface elements attests to the effectiveness of the design and implementation.

3.2.4 Error Handling and Resilience

In the development of the pedestrian crossing system, particular attention was given to error handling and resilience. Various mechanisms were implemented to ensure the robustness of the system in the face of unexpected events or faults.

To enhance system resilience, the software employs a real-time operating system (FreeRTOS). The use of tasks and priorities allows the system to handle multiple concurrent activities efficiently. Tasks related to different functionalities, such as traffic light control and SPI communication, operate independently, reducing the impact of potential failures in one area on the overall system.

3.2.5 Documentation

Thorough documentation is crucial for understanding and maintaining the codebase. The project follows the Doxygen standard to ensure clarity and completeness in code documentation.

Every source code file includes a header that provides a high-level overview of the file's purpose and functionality. Additionally, each function within the codebase is meticulously documented using Doxygen comments. These comments include information on the function's purpose, parameters, return values, and any potential side effects.

In addition to code documentation, the project includes comprehensive documentation on the hardware setup and overall system architecture. The hardware block diagram Figure 1 visually represents the connections between different components, aiding in understanding the physical layout of the system.

Moreover, the software layered diagram Figure 2 provides insights into the high-level structure of the software, illustrating the relationships between different software modules and their interactions.

4 Conclusion

Throughout the project, various challenges were encountered, providing valuable learning experiences. The iterative nature of development, especially in the state machine implementation, highlighted the importance of adaptability and persistence. Overcoming challenges such as flickering LEDs during RTOS implementation demonstrated the significance of thorough debugging and optimization.

As with any project, there is always room for improvement. Future iterations could explore alternative code structures for the state machine, potentially enhancing readability and maintainability. Additionally, investigating power-saving mechanisms during idle periods could contribute to the overall efficiency of the system.

4.1 Compliance with Objectives

Both Task 1 and Task 2 were successfully implemented and are functioning together. Moreover, all the requirements for Task 3 were met as outlined in the project description [2]. However, the teacher did not provide an automated grading system to verify our work. As a result, I did not have a method to be completely confident that I implemented all the tasks. Nevertheless, based on my understanding and interpretation of the task descriptions, I believe all the tasks were implemented.

4.2 Challenges

Several challenges arose during the development. For instance, I thought that there were quite many requirements that had to be met for Task 3. Getting everything to work together was quite challenging and the code had to be rewritten several times. For example, the state machine code was rewritten at least 3 times. I also had problems with the LEDs when implementing RTOS because it split the update LEDs task into a separate task. The task was running too fast and often, causing the LEDs to flicker. I solved it by only updating the LEDs when a change in the LEDs data variable had been updated. In this way, the LEDs stop flickering because they are only updated when a change has occurred.

4.3 Possible improvements

There is always room for improvement, whether it's making the code cleaner or implementing a functionality in a different way. One improvement that comes to mind is the code structure of the state machine. Currently, the state machine is made up of multiple nested if statements, perhaps the program could be structured in a different way.

Perhaps implement some type of low power mode when no cars or pedestrians are present to save on power. While this was not mandatory, it could be useful to implement in the real-world application of the traffic light system, especially if the system has power constraints.

4.4 Final Thoughts

Embarking on the development of the pedestrian crossing system has been both educational and enjoyable. The project provided a hands-on application of theoretical knowledge gained in the Embedded Systems course IS1300, offering insights into hardware configurations, C programming, and practical aspects of FreeRTOS.

In conclusion, the pedestrian crossing system project encapsulates not just technical learning but an adventure of discovery and growth. The gained knowledge and experiences serve as a solid foundation for future endeavors in the dynamic field of embedded systems.

5 References

References

[1] M. Becker. (2023) Schematics page. Accessed: December 17, 2023. [Online]. Available: https://canvas.kth.se/courses/43011/pages/schematics?module_item_id=761472

[2] ——. (2023) Project description. Accessed: December 17, 2023. [Online]. Available: https://canvas.kth.se/courses/43011/pages/project-description?module_item_id=761471